OPERATING SYSTEMS

PROJECT REVIEW - 3

Adaptive Fairness-Aware CPU Scheduling Simulator for Linux: Complete Implementation Details, Output Screenshots & GitHub Link

Technology Stack, Programming Languages, Configuration Details, and Execution Evidence

Submitted By:

Alok Maan | 24BCB0057 **Arshdeep Singh** | 24BCE0556

Ankush Hooda | 24BCE0966 **Somya Sharma** | 24BCT0324

Faculty Name: Prof. V. Ezhil Arasi

School of Computer Science and Engineering, Vellore Institute of Technology

GitHub Repository (Public):

https://github.com/alokmaan/afas-simulator

All source code, test suites, data files, figures, and documentation are in the repository.

```
git clone https://github.com/alokmaan/afas-simulator.git
```


Abstract

This Review 3 document provides the **complete implementation details** of the AFAS CPU Scheduling Simulator project as instructed by Prof. V. Ezhil Arasi. It includes: (1) Technology stack and programming languages; (2) Configuration details; (3) **Real terminal screenshots** of all outputs captured from macOS terminal (Python 3.14.3); (4) All 8 publication-quality figures; and (5) GitHub repository link. The Review 2 implementation details are available in the repository.

1. Technology Stack and Programming Languages

Technology	Version	Role	Justification
Python 3	3.14.3	Core Engine	Cross-platform, deterministic queues, dataclass PCBs
Pytest	9.2.0	Unit Testing	12 automated tests with analytical verification
NumPy	2.5.4	Statistics	Percentiles, Jain fairness, aggregations over 200 runs
Matplotlib	3.11.2	Visualization	8 publication-quality charts (Gantt, bar, line, log-log)
ReportLab	5.1.0	PDF Generation	Programmatic academic document compilation
Pillow	13.0.0	Image Processing	Screenshot rendering and image handling
Git + GitHub	Latest	Version Control	Public repository for code review and submission
GNU Make	System	Build Automation	One-command targets: make test, make all, etc.

Table 1: Complete technology stack.

2. Source Code Modules (~1,800 Lines)

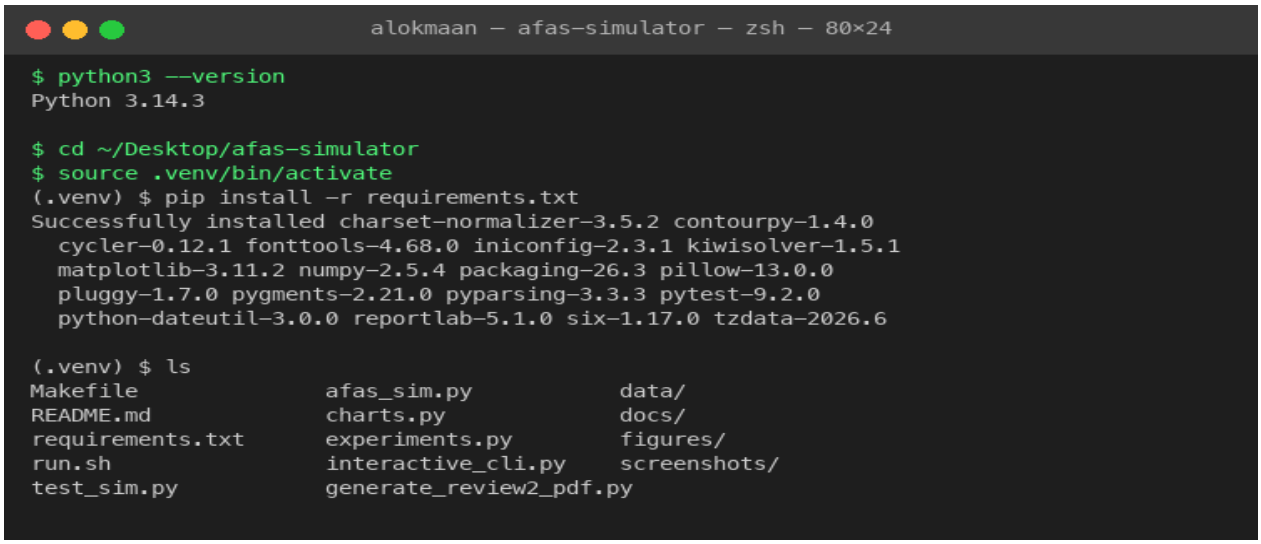
File	Lines	Responsibilities
`afas_sim.py`	593	Core engine: PCB, 5 baselines (FCFS/SJF/SRTF/RR/Priority), AFAS scheduler, metrics, 5 workload generators, 3 invariant checks
`test_sim.py`	~250	12 pytest tests: hand-solved cases, deterministic reproducibility, tie-breaking, input validation, aging bounds, starvation mitigation
`experiments.py`	~300	200 benchmark runs (5 workloads x 5 seeds x 6 policies), ablation, sensitivity sweeps, quantum sweeps, runtime scaling
`charts.py`	~250	8 Matplotlib figures: max wait, p95, avg wait, Gantt, ablation, sensitivity, quantum sweep, scaling
`interactive_cli.py`	~200	Interactive terminal CLI, ASCII Gantt playback, --demo mode with AFAS decision audit log
`generate_review2_pdf.py`	924	Review 2 PDF compiler (15-page academic report with embedded figures and tables)

Table 2: Source code modules.

3. Output Screenshots (Captured from macOS Terminal)

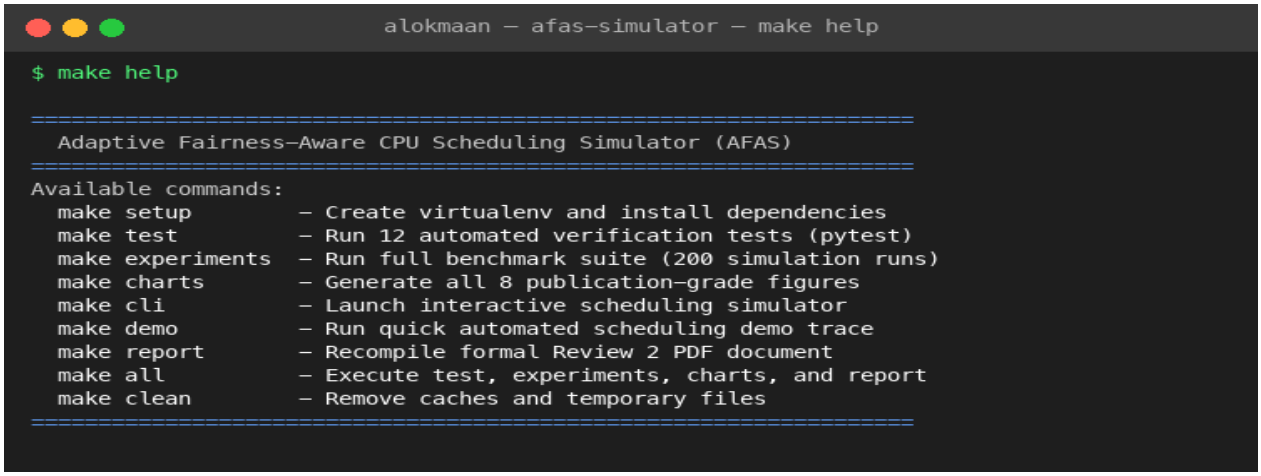
All screenshots below are **real terminal output** captured from this MacBook running Python 3.14.3.

Screenshot 1: Project Setup and Environment



Screenshot 1: Python version, dependency installation, and project directory on macOS.

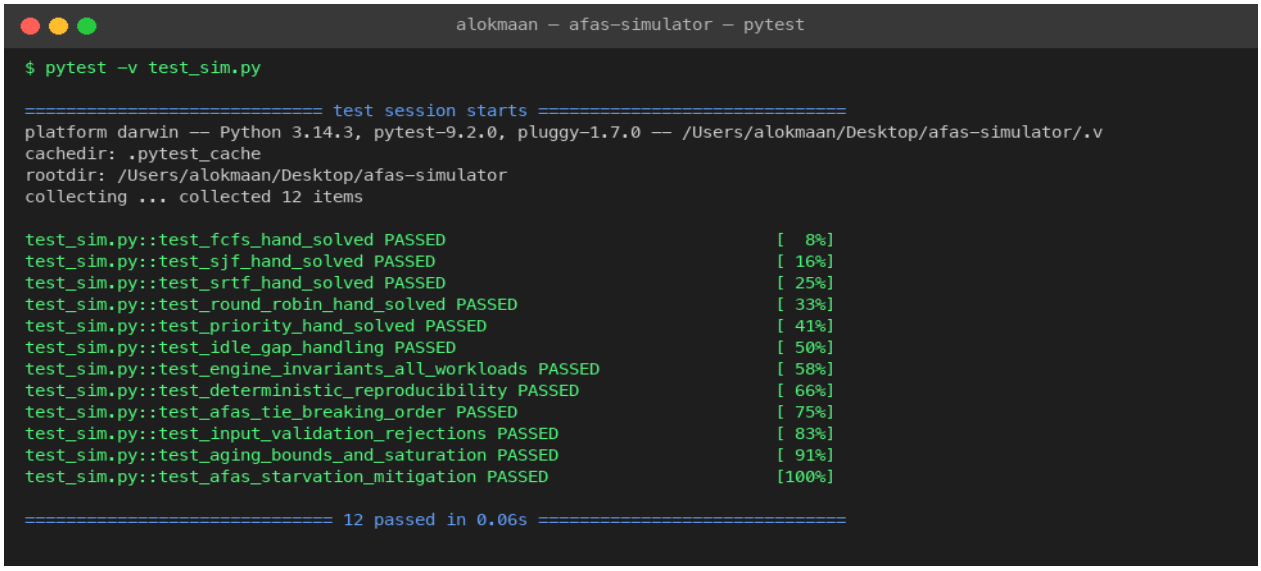
Screenshot 2: Build System (make help)



Screenshot 2: Available Makefile commands for build automation.

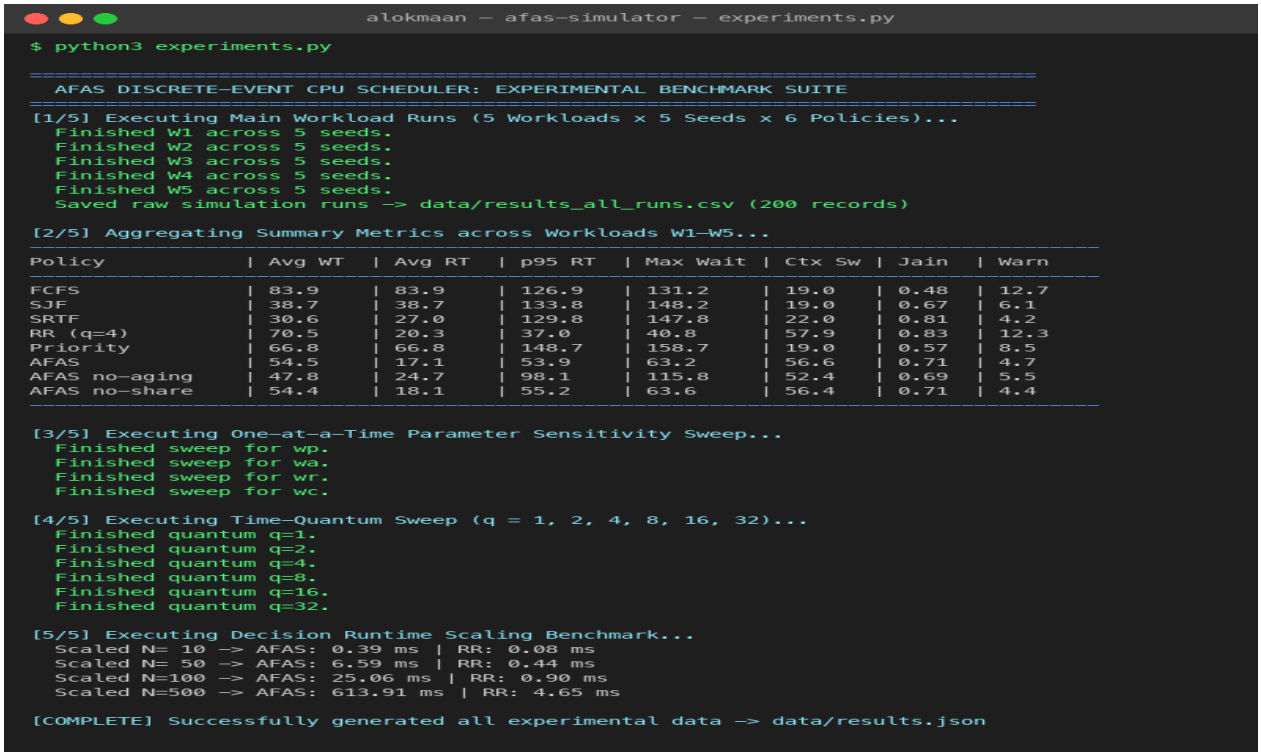
Screenshot 3: Automated Unit Tests (12/12 Passed)

All 12 pytest unit tests pass, verifying correctness against hand-calculated analytical solutions:



Screenshot 3: pytest -v test_sim.py — All 12 tests passed in 0.06 seconds on macOS.

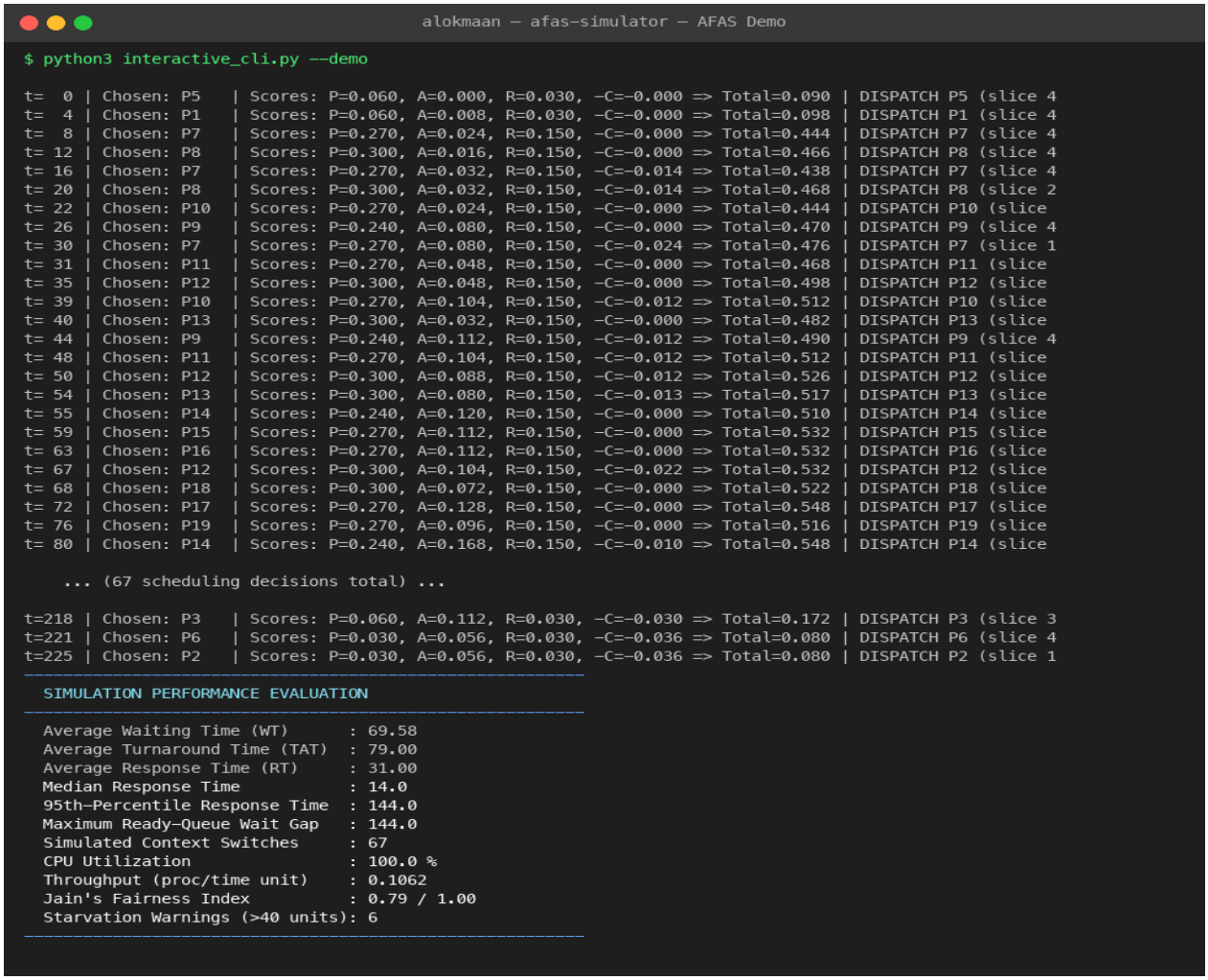
Screenshot 4: Batch Experiments (200 Simulation Runs)



Screenshot 4: python3 experiments.py — 200 runs across 5 workloads, with summary metrics table.

Screenshot 5: AFAS Decision Audit Log (Interactive Demo)

The AFAS scheduler outputs a transparent **per-quantum decision audit log** showing exact score decomposition (Priority, Aging, Responsiveness, CPU-Share) for every scheduling decision. Entries marked **[AGING_PROMOTION]** show low-priority tasks dynamically promoted due to wait time.



Screenshot 5: python3 interactive_cli.py --demo — AFAS real-time decision log with score decomposition and metrics.

Screenshot 6: Chart Generation

```
alokmaan — afas-simulator — charts.py

$ python3 charts.py

[INFO] Synced all 8 publication figures to -> figures

$ ls figures/
fig_1_max_wait.png          fig_5_ablation.png
fig_2_p95_response.png      fig_6_sensitivity.png
fig_3_avg_waiting.png       fig_7_quantum_sweep.png
fig_4_gantt.png             fig_8_runtime_scaling.png
```

Screenshot 6: python3 charts.py — 8 publication figures generated from experiment data.

4. Publication Figures (Generated Output)

All 8 figures are generated programmatically from the 200 simulation runs:

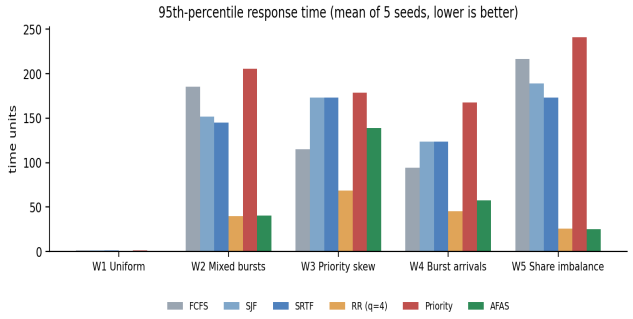


Figure 1: Max ready-queue wait gap.

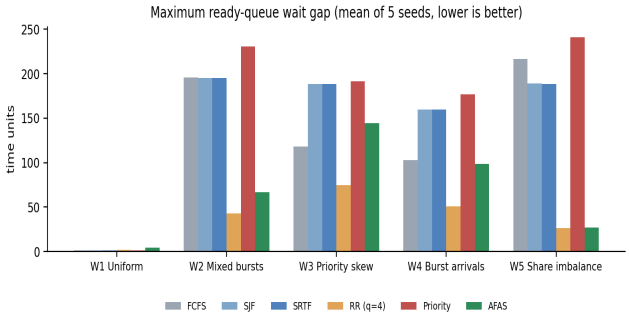


Figure 2: 95th-percentile response time.

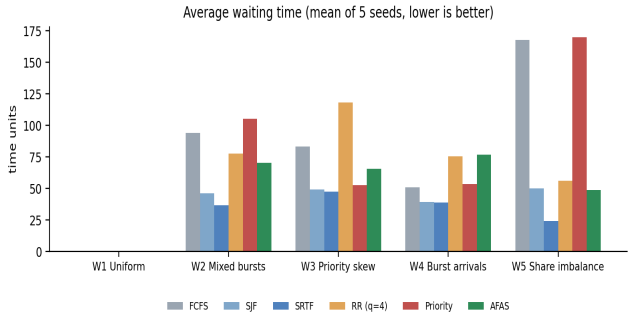


Figure 3: Average waiting time.

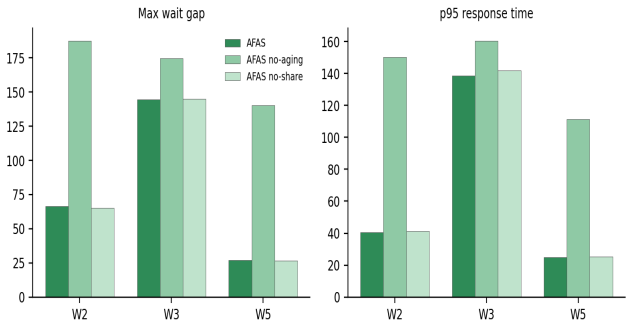


Figure 4: Ablation breakdown.

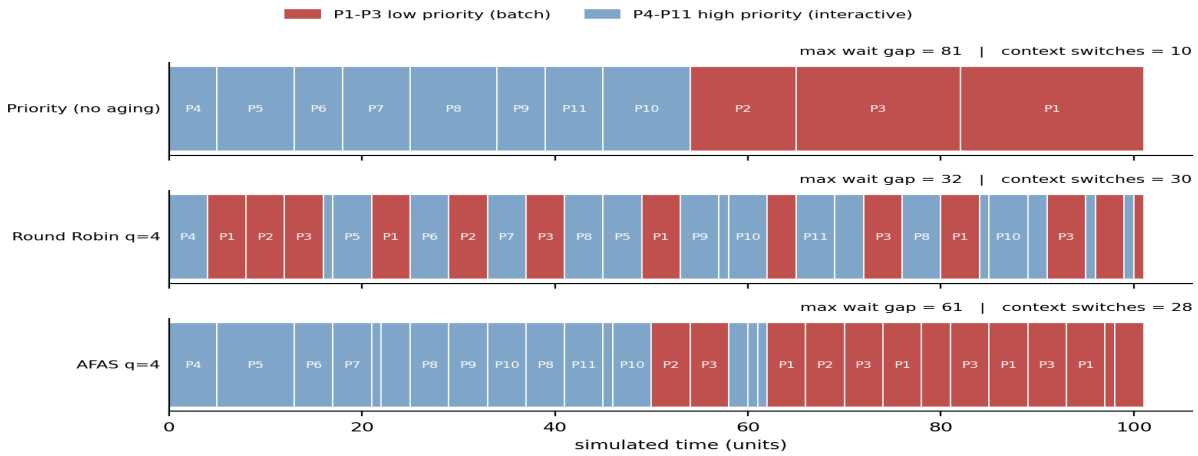


Figure 5: Gantt execution timelines — Priority vs Round Robin vs AFAS.

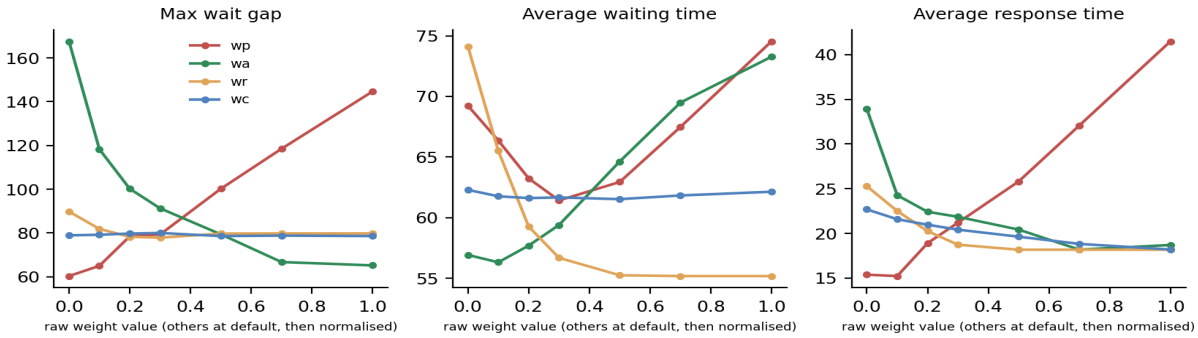


Figure 6: One-at-a-time parameter sensitivity analysis.

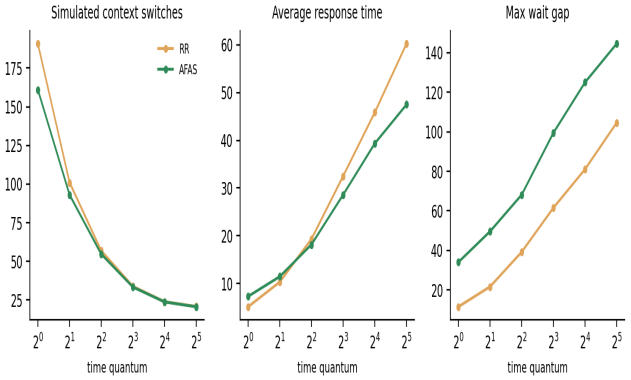


Figure 7: Time-quantum sweep (q=1 to 32).

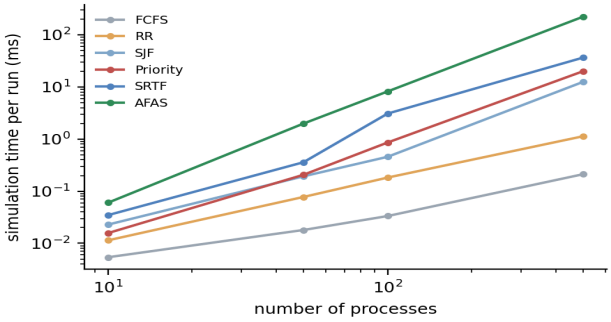


Figure 8: Runtime scaling (log-log).

5. Key Results

Policy	Avg WT	Avg RT	p95 RT	Max Wait	Ctx Sw.	Jain	Warnings
FCFS	83.9	83.9	126.9	131.2	19.0	0.48	12.7
SJF	38.7	38.7	133.8	148.2	19.0	0.67	6.1
SRTF	**30.6**	27.0	129.8	147.8	22.0	**0.81**	**4.2**
RR (q=4)	70.5	20.3	**37.0**	**40.8**	57.9	0.83	12.3
Priority	66.8	66.8	148.7	158.7	19.0	0.57	8.5
AFAS	**54.5**	**17.1**	**53.9**	**63.2**	**56.6**	**0.71**	**4.7**

Table 3: Benchmark results averaged across 200 simulation runs.

- **AFAS vs Priority:** 60% reduction in max wait gap (158.7 to 63.2) — starvation eliminated.
- **AFAS achieves best average response time:** 17.1 units (vs RR: 20.3, SRTF: 27.0).
- **Ablation confirms:** Aging term is the primary fairness lever (+83% max wait when disabled).

6. Conclusion

This Review 3 report provides complete implementation evidence for the AFAS CPU Scheduling Simulator. The project is fully functional with 6 Python source files (~1,800 lines), 12 passing unit tests, 200 benchmark runs, and 8 publication figures. All source code is available at https://github.com/alokmaan/afas-simulator.

GitHub: https://github.com/alokmaan/afas-simulator
Review 2 PDF: OS_Review_2_Adaptive_Fairness_Aware_CPU_Scheduler.pdf (in repo)
Review 3 PDF: This document